



3. Conhecendo a linguagem

☒ Dúvidas?	☐
☒ Exercícios a entregar?	☒
 Módulo	1 - Primeiros passos com Python e Versionamento de código
 Tipo de aula	Teste Vídeo
 Links e mídias	Pasta "Aula 3" https://github.com/digitalinnovationone/trilha-python-dio
 Última atualização	@17 de novembro de 2025 03:58
 Status	Done
 Data início	@06/11/2025
 Data conclusão	@06/11/2025 15:14
 Esforço	Fácil

Anotações

Vídeo 1 - Tipos de dados

Etapa 1: O que são tipos?

Etapa 2: Tipos numéricos

Etapa 3: Booleanos e strings

Vídeo 2 - Modo interativo

Etapa 1: Usando o modo interativo

Etapa 2: Funções `dir` e `help`

Vídeo 3 - Variáveis e constantes

Etapa 1: O que são variáveis e constantes?

Etapa 2: Práticas recomendadas

Vídeo 4 - Conversão de tipos

Inteiro para float

Float para inteiro

Conversão por divisão

Números para strings

String para número

Erros de conversão

Vídeo 5 - Funções de entrada e saída

Etapa 1: Ler valores com a função `input`

Etapa 2: Exibir valores com a função `print`

Exercícios

Arquivos

Anotações

Vídeo 1 - Tipos de dados

Etapa 1: O que são tipos?

Para esta aula não serão considerados os tipos criados pelo usuário, mas sim os padrões da linguagem:

- Texto: `str`
- Numéricos: `int`, `float`, `complex`
- Sequência: `list`, `tuple`, `range`
- Mapa: `dict`
- Coleção: `set`, `frozenset`
- Booleano: `bool`
- Binário: `bytes`, `bytearray`, `memoryview`

O menos utilizado dos tipos mencionados acima é `complex`, porque geralmente se usa `int` ou `float`.

Etapa 2: Tipos numéricos

Os números inteiros, representados pela classe `int`, têm precisão ilimitada. Exemplos de números válidos: 1, 10, 100, -1, -10, -100... 99001823...

Os números de ponto flutuante, representados pela classe `float`, indicam os números racionais. Exemplos de números válidos: 1.5, -10.543, 0.76... 999278.002...

Etapa 3: Booleanos e strings

Booleanos são usados para representar os valores `True` (verdadeiro) ou `False` (falso), implementados pela classe `bool`. Essa classe é uma subclasse de `int`, porque qualquer número diferente de `0` representa `True` e `0` representa `False`.

OBSERVE que as palavras "`True`" e "`False`" PRECISAM ser escritas com a primeira letra EM MAIÚSCULA.

Strings, ou cadeias de caracteres, indicam valores alfanuméricos, e são representados pela classe `str`. Exemplos válidos de strings: "Python", 'Python', """Python""", """"Python""", "", "p".

Vídeo 2 - Modo interativo

Etapa 1: Usando o modo interativo

O modo interativo é o interpretador Python, que possibilita escrever o código e ver o resultado na hora. Para iniciar o modo interativo, podemos:

- Chamar o interpretador, com `python3`
- Executar o código com a *flag* `-i` (`python3 -i app.py`)

Etapa 2: Funções `dir` e `help`

`dir`, sem argumentos, retorna a lista de nomes no escopo local atual. Com um argumento, retorna uma lista de atributos válidos para o objeto. Exemplos:

```
dir() # retorna ['PS1', 'REPLHooks', '__annotations__', '__builtins__', '__doc__',
'__loader__', '__name__', '__package__',
        # '__spec__', 'get_last_command', 'is_wsl', 'original_ps1', 'platform', 'r
eadline', 'sys']
```

```

dir(100) # retorna ['__abs__', '__add__', '__and__', '__bool__', '__ceil__', '__clas
s__', '__delattr__', '__dir__',
                # '__divmod__', '__doc__', '__eq__', '__float__', '__floor__', '__floor
div__', '__format__', '__ge__',
                # '__getattr__', '__getnewargs__', '__getstate__', '__gt__', '__
hash__', '__index__', '__init__',
                # '__init_subclass__', '__int__', '__invert__', '__le__', '__lshift__', '__l
t__', '__mod__', '__mul__',
                # '__ne__', '__neg__', '__new__', '__or__', '__pos__', '__pow__', '__r
add__', '__rand__', '__rdivmod__',
                # '__reduce__', '__reduce_ex__', '__repr__', '__rfloordiv__', '__rlshi
ft__', '__rmod__', '__rmul__', '__ror__',
                # '__round__', '__rpow__', '__rrshift__', '__rshift__', '__rsub__', '__rt
ruediv__', '__rxor__', '__setattr__',
                # '__sizeof__', '__str__', '__sub__', '__subclasshook__', '__truediv_
__', '__trunc__', '__xor__',
                # 'as_integer_ratio', 'bit_count', 'bit_length', 'conjugate', 'denomi
nator', 'from_bytes', 'imag', 'is_integer',
                # 'numerator', 'real', 'to_bytes']

```

Se dermos o comando `import math`, o resultado de `dir` muda:

```

['PS1', 'REPLHooks', '__annotations__', '__builtins__', '__doc__', '__loader__',
 '__name__', '__package__',
 '__spec__', 'get_last_command', 'is_wsl', 'math', 'original_ps1', 'platform', 're
adline', 'sys']

```

help chama o sistema de ajuda integrado. Podemos fazer buscas no modo interativo ou informar o nome do módulo, classe, função, método ou variável como um parâmetro:

```

help() # abre o modo interativo no terminal e exibe a seguinte mensagem:
        # Welcome to Python 3.13's help utility! If this is your first time using
        # Python, you should definitely check out the tutorial at
        # https://docs.python.org/3.13/tutorial/.
        # Enter the name of any module, keyword, or topic to get help on wr
iting
        # Python programs and using Python modules. To get a list of avail

```

```
able
    # modules, keywords, symbols, or topics, enter "modules", "keywor
ds",
    # "symbols", or "topics".
help(100) # exibe a implementação da classe do parâmetro indicado entre
parênteses. É o mesmo que pressionar Ctrl e
    # clicar na classe.
```

Vídeo 3 - Variáveis e constantes

Etapa 1: O que são variáveis e constantes?

Em linguagens de programação, podemos definir valores que podem sofrer alterações no decorrer da execução do programa. Esses valores recebem o nome de **variáveis**, pois nascem com um valor e não necessariamente permanecem com o mesmo durante a execução do programa.

Note que, ao atribuir um valor a uma variável, o Python estabelece seu tipo de dado, portanto **não é possível** alterar o tipo de dado sem alterar o valor da variável.

Assim como as variáveis, constantes armazenam valores, porém permanece com o mesmo valor até o final da execução do programa. Seu valor é imutável.

Entretanto, devemos lembrar que, em Python, não há uma palavra reservada que define constantes. Para fazer essa definição, a prática recomendada é colocar o nome da variável **todo em LETRAS_MAIÚSCULAS**. Alguns exemplos de coisas que devem ser constantes: nomes de pessoas, estados, cidades e países, caminhos absolutos, valores específicos, um modo de depuração ou de execução...

Etapa 2: Práticas recomendadas

- Para definir uma variável de valor constante, a prática recomendada é colocar o nome da variável **todo em LETRAS_MAIÚSCULAS**.
- O padrão de nomes deve ser todo em minúsculas (`snake_case`), com o sublinhado separando as palavras. Lembrando que mesmo as constantes devem seguir esse padrão: `LETRAS_MAIÚSCULAS`.
- Deve-se escolher nomes sugestivos para variáveis; de nada adianta nomear uma variável como `p`, `ax` ou `lim_saq_d`.

Vídeo 4 - Conversão de tipos

Como converter tipos de variáveis, por exemplo: converter uma variável do tipo string para número para poder usá-la em algum cálculo.

Inteiro para float

```
preco = 10 # retorna 10
preco = float(preco)
print(preco) # retorna 10.0
```

Float para inteiro

```
preco = 10.3 # retorna 10.3
preco = int(preco)
print(preco) # retorna 10
```

Conversão por divisão

Outra forma de fazer uma conversão é usando os operadores de divisão (`/`) ou (`//`):

```
valor = 10 # retorna 10
valor = 10/2 # retorna 5.0
valor = 10//2 # retorna 5.0
```

Números para strings

```
preco = 10.50
idade = 28
print(str(preco))
print(str(idade))
```

String para número

```
preco = "10.50"
idade = "28"
```

```
print(float(preco))  
print(int(idade))
```

Erros de conversão

```
preco = "python"  
print(float(preco))  
# retorna "ValueError: could not convert string to float: 'python'"
```

Vídeo 5 - Funções de entrada e saída

Etapa 1: Ler valores com a função `input`

Usamos a função incorporada `input` quando precisamos ler dados inseridos pelo teclado. Ela recebe um argumento do tipo `str`, que é exibido para o usuário na tela. A função lê a inserção, converte o valor para uma string e retorna o valor.

Etapa 2: Exibir valores com a função `print`

Usamos a função incorporada `print` quando precisamos exibir dados na tela. Ela recebe um argumento obrigatório do tipo `varargs` de objetos e 4 argumentos opcionais (`sep`, `end`, `file` e `flush`). Todos os objetos são convertidos para string, separados por `sep` e terminados por `end`. A string final é exibida para o usuário.

Exercícios

100% - 10/10 corretas

Arquivos

Pasta "Aula 3"

<https://github.com/digitalinnovationone/trilha-python-dio>